



An analysis of lock-based and optimistic concurrency control protocols in multiprocessor real-time databases

Anthony Chiu^a, Ben Kao^{a,*}, Kam-yiu Lam^b

^a Department of Computer Science, The University of Hong Kong, Pokfulam, Hong Kong

^b Department of Computer Science, City University of Hong Kong, 83 Tat Chee Avenue, Hong Kong

Received 1 July 1997; received in revised form 1 November 1997; accepted 1 January 1998

Abstract

Previous studies, e.g., Haritsa et al. (Haritsa, J.R., Livny, M., Carey, M., 1990. Proceedings of Ninth ACM Symposium on Principles of Database systems) have shown that optimistic concurrency control (OCC) generally performs better than lock-based protocols in disk-based real-time database systems (RTDBS). In this paper we compare the two concurrency control protocols in both *disk-based* and *memory-resident* multiprocessor RTDBS. Based on simulation, we analyze the intrinsic behaviors of the two protocols. The result of our performance evaluation experiments show that different characteristics of the two environments indeed have great impact on the protocols' performance. We identify such system characteristics and expose the weaknesses of traditional OCC and lock-based protocols. To improve performance, a new protocol, called Two Phase Locking-Lock Write All (2PL-LW), is proposed. We show that 2PL-LW performs better than the traditional protocols in meeting transaction deadlines in both disk-based and memory-resident RTDBS. © 1998 Elsevier Science Inc. All rights reserved.

Keywords: Real-time databases; Concurrency control; Multiprocessor; Transaction scheduling

1. Introduction

Real-time database systems (RTDBS) are database systems whose transactions are associated with timing constraints such as *deadlines*. The performance and correctness of an RTDBS depend on how well these constraints are met. For example, an electronic program trading application may respond to a stock's price change by spawning a transaction to calculate a good holding strategy. This transaction needs to be completed by a certain deadline before the market condition is shifted too wide from the original price quotes. As another example, a radar surveillance system detects aircraft "images" or "radar signatures." These images are then matched against a database of known images. The result of such a match is used to drive other system actions, for example, in choosing a combat strategy.

Besides meeting transaction timing constraints, an RTDBS needs to observe data consistency constraints as well. A number of studies have been done on this subject, e.g., (Abbott and Garcia-Molina, 1988; Haritsa

et al., 1990, 1991, 1993a, b; Huang et al., 1992; Ulusoy, 1994), in which different algorithms and protocols are suggested to schedule transactions according to their priorities. For example, the Earliest-Deadline-First (EDF) algorithm assigns a higher priority to a transaction with an earlier deadline; the High-Priority (HP) concurrency control protocol allows a higher priority transaction abort a lower priority one when they conflict in accessing common data.

In this paper we study the performance of two prevalent concurrency control protocols in both *disk-based* and *memory-resident multiprocessor real-time database systems*, namely, *Two-Phase Locking-High Priority* (2PL-HP) (Abbott and Garcia-Molina, 1988) and *Optimistic Concurrency Control-Broadcast Commit* (OCC-BC) (Menasce and Nakanishi, 1982; Robinson, 1982). The goal is to present an in-depth study of the intrinsic advantages and disadvantages of these two protocols. For example, optimistic (or validation) protocols were originally proposed to reduce system overhead and to improve performance for conventional database systems with *mostly read* transactions (Korth and Silberschatz, 1986). However, one of our findings shows that OCC also outperforms 2PL in *disk-based real-time* databases with mostly write transactions as well, given that meeting

* Corresponding author. Tel.: +852 2857 8445; fax: +852 2559 8447; e-mail: kao@cs.hku.hk.

transaction deadlines is the primary performance indicator. As we will see later in this paper, the phenomenon is a result of an implicit load control mechanism brought along by OCC. As another example of our findings, the differences in the access characteristics between disk and memory have a tremendous impact on the protocols' performance. These findings enable us to engineer a better protocol that works well in both disk-based and memory-based systems.

The problem of 2PL-HP vs. OCC-BC in a disk-based system has been carefully studied in (Haritsa et al., 1990). In that paper, it is shown that OCC-BC outperforms 2PL-HP under various system settings in disk-based RTDBS. One major reason is that under 2PL-HP, a (higher priority) transaction that restarts a (lower priority) one may later be restarted or miss its deadline due to the long and unpredictable disk access delay. This results in many *useless restarts* (Haritsa et al., 1990) and thus poor utilization of system resources. In this study we revisit the 2PL-HP vs. OCC-BC problem and investigate how data access delay affects the performance of the concurrency control protocols. The motivations behind this study are:

- Disk accesses are relatively slow and cause much unpredictability to transaction response time. Also, disk characteristics are particularly defiant to real-time scheduling. For example, serving disk requests EDF causes poor disk utilization and performance (Abbott and Garcia-Molina, 1990). High performance real-time database systems, therefore, tend to move *real-time data* – those accessed by transactions with stringent timing constraints – into main memory for fast, predictable, and parallel accesses (Huang, 1993). It is therefore useful to study the concurrency control protocols in a memory-resident RTDBS.
- Although optimistic concurrency control is shown to perform better in disk-based real-time systems, most commercially available database systems use two-phase locking. It is thus important to understand the intrinsic inefficiency of 2PL-HP and the inherent advantages of OCC-BC in a disk-based system. The analysis allows us to improve the more commonly used lock-based protocols to attain a better performance.

The rest of this paper is organized as follows. In Section 2 we give a brief description and comparison of 2PL-HP and OCC-BC. We highlight the characteristics of the two concurrency control protocols that contribute to their different behaviors in disk-based and memory-resident systems. Based on the analysis, we propose, in Section 3, a variant of 2PL-HP–2PL-LW which has the advantages of both 2PL-HP and OCC-BC. To compare the protocols' performance, we perform a thorough simulation study. Section 4 describes our real-time database system model and the simulation model. Results of the simulation experiments are presented in Section 5.

In the practical implementation of OCC-BC, blocking is possible at the start of a transaction in order to preserve data integrity. This form of blocking, as we observe through our experimentation, imposes an indirect load control mechanism that regulates the population of active transactions in the system. This effect is particularly important in avoiding system thrash when the database system is under a sudden load stress. In Section 6 we investigate the advantage of such a load control mechanism and show how 2PL-LW can be enhanced to incorporate such control to attain a higher performance. Finally, Section 7 concludes the paper.

2. Concurrency control protocols

In a database system, transactions interact with each other through reads and writes of data pages. Concurrency control protocols are designed to maintain the database consistency despite concurrent execution of transactions (Bernstein et al., 1987). For RTDBS, these conventional protocols are unsatisfactory because they do not take transactions' priorities into account. For example, under Two-Phase Locking (2PL), a transaction holding a lock could have a lower priority than (and thus block) a high-priority requester – a phenomenon called *priority inversion* (Abbott and Garcia-Molina, 1988). Since the low priority lock holder is discriminated against in its use of system resources (e.g., the CPU), the blocked high-priority transaction will experience extensive delay and is likely to miss its deadline. Applying the concurrency control protocols to an RTDBS thus requires modifications to the basic methods in resolving data conflicts. In the rest of this section, we mention two approaches: 2PL-HP (Section 2.1) and OCC-BC (Section 2.2). We analyze their advantages and disadvantages. In Section 3, we propose a new lock-based protocol *Two-Phase Locking – Lock Writes All* (2PL-LW) which adopts the advantages of both 2PL-HP and OCC-BC.

2.1. Two-phase locking-high priority

The basic Two-Phase Locking protocol is the most common locking protocol in conventional database systems. With 2PL, a transaction execution consists of two phases. In the first phase, locks are acquired but may not be released. In the second phase, locks are released but new locks may not be acquired. In case a transaction T_R requests a lock that is being held by another transaction, T_H , T_R waits.

As we have just demonstrated, one basic problem of 2PL is the possibility of priority inversions. One solution to this problem is to restart the low-priority lock holder and let the high-priority lock requester proceed. This variant of 2PL is called 2PL-HP (Abbott and

Garcia-Molina, 1988). Conflicts are thus resolved by a combination of blocking and restarts under 2PL-HP.

2.2. Optimistic concurrency control-broadcast commit

Although most commercially available database systems use lock-based concurrency control protocols, recent studies (Haritsa et al., 1990; Huang and Stankovic, 1993) have suggested that optimistic concurrency control protocols (OCC) provide better performance in RTDBS. Here, we describe a variant of OCC called Optimistic Concurrency Control with Broadcast Commit (OCC-BC) (Menasce and Nakanishi, 1982; Robinson, 1982).

Under OCC-BC, the execution of a transaction is divided into three phases: (1) read phase, (2) validation phase and (3) write phase. During the read phase, data pages are read into memory. Computations based on the values of these data pages are performed. New values are computed, but are not written into the database until the write phase. When a transaction T finishes its computation, it enters the validation phase in which all transactions that conflict with T are restarted. (This is done by checking the read-sets and the write-sets of transactions.) Finally, during the write phase, updated data pages are written back to the database.

Although in principle, OCC-BC resolves conflicts *mainly* through restarting transactions, in practice, transactions do suffer waits (or blocking) occasionally. To see this, consider a transaction T_1 that has just passed its validation phase. Suppose before T_1 starts its write phase another transaction T_2 enters the system and reads a page to be written by T_1 , T_2 will miss T_1 's write. The transactions schedule is thus not serializable. One solution to this race condition is to mark the data pages to be written by T_1 as “busy” during T_1 's write phase. Transaction T_2 will thus be blocked before it tries to read the “busy” pages. This strategy guarantees that as long as a transaction reaches its validation phase, it will not be restarted by or waits for others. OCC-BC thus also carries a blocking factor in its conflict resolution scheme.

2.3. Performance comparison: 2PL-HP vs. OCC-BC

To compare the performance of 2PL-HP and OCC-BC under both disk-based and memory-resident systems, we need to understand the characteristics of the two concurrency control protocols and the characteristics of the two storage environments. Here, let us first identify three characteristics of a concurrency control protocol which directly affects its performance in an RTDBS.

Number of restarts: Both 2PL-HP and OCC-BC use transaction restarts as a part of their conflict resolution strategies. Restarting a transaction means giving up the

resources already invested into the transaction and is thus detrimental to the system's performance. In OCC-BC, a restart will occur whenever there are conflicts between the read-sets and the write-sets of two concurrent transactions. Comparing with the part-blocking, part-restarting conflict resolution strategy of 2PL-HP, OCC-BC generates more restarts than 2PL-HP does and thus wastes more system resources than 2PL-HP.

Useless restarts: After a transaction T_1 restarts another transaction T_2 , T_1 may later miss its deadline. We call the restart of T_2 *useless*, since this action does not help T_1 make its deadline. Under 2PL-HP, T_1 may miss its deadline (and thus causes a useless restart) because of the delay experienced in further resource contention (CPU and/or I/O), or because it is later restarted by or waits for another higher priority transaction during conflict resolution. Under OCC-BC, a transaction that restarts others must be committing, and thus will never be restarted by or waits for others. In this respect, OCC-BC is better than 2PL-HP in avoiding useless restarts.¹

Useless Waits: In 2PL-HP, when a transaction T_1 is blocked by another transaction T_2 due to lock conflict, T_1 is delayed. If T_2 later becomes tardy, the wait caused to T_1 is *wasted*. We call the blocking caused by T_2 *useless wait*. Similar to the arguments for *Useless Restarts*, under 2PL-HP, T_2 may miss its deadline (and thus causes a useless wait) because of further resource contention (CPU and I/O) and/or conflict resolution. On the other hand, under OCC-BC, a transaction that blocks others must be committing and thus will never be restarted by or waits for others. Hence, OCC-BC is better than 2PL-HP in avoiding useless waits.

We would like to remark that while a restart clearly forgoes some system resources invested in the restarted transaction and thus undermines the capacity of the system, a waiting transaction does not burn much resources. As long as the transaction has enough amount of slack to absorb the wait time, it can make its deadline. However, in general, useless waits do not help any transaction. It is undesirable and should be avoided.

We summarize the advantages and disadvantages of 2PL-HP and OCC-BC in Table 1. In the table, we use a ✓ (×) to indicate good (bad) performance with respect to a certain system characteristic.

Note that we have only identified the performance-affecting factors. The extent of the performance impact that each of them carries depends on the system parameters, such as system load, data access time, and the degree of data contention among transactions. We will

¹ If buffer management is not used, OCC-BC does not prevent useless restarts completely. This is because after the validation phase, new data still needs to be written to the physical database before the deadline. Storage access delay may still cause the transaction to become tardy.

Table 1
Relative advantages and disadvantages of 2PL-HP and OCC-BC

	2PL-HP	OCC-BC
Number of restarts	✓	×
Useless restarts	×	✓
Useless waits	×	✓

show our simulation analysis on these factors in Section 5. We next proceed with a discussion on the difference between disk accesses and memory accesses and the implication of it on the different behaviors of the two protocols.

First, disk accesses are slow and unpredictable. The response time of a disk request depends on the position of the disk head (and thus on the previous disk access). Typical disk access time is in the order of milliseconds with a relatively large variance. Moreover, disk requests are served one at a time. Therefore, it is possible that a disk request (and the issuing transaction) is blocked by a few others due to *disk conflict* (even if there is no data conflict among the transactions). Disk access delay and, more importantly, its variance, are amplified by the *chain of requests*. These factors are detrimental to satisfying transaction deadlines. Memory accesses, on the other hand, are pretty constant. Apart from data locking, data requests from different transactions can be served in parallel (Huang, 1993). Blocking is thus not induced by data accesses in memory-resident system.

In Haritsa et al. (1990), it is found that OCC-BC outperforms 2PL-HP in disk-based RTDBS. However, as we will see in our simulation analysis (Section 5), 2PL-HP performs better than OCC-BC in memory-resident RTDBS. Here, let us briefly explain this observation.

Recall that there are three adverse factors affecting the performance of the concurrency control protocols (Table 1). For example, 2PL-HP is good because it generates fewer restarts, but it is bad because more restarts and waits are useless. Whether OCC-BC or 2PL-HP is the better protocol depends on the seriousness of these factors.

As we have just mentioned, data access in a disk-based system could be long, and with a large variance in response time. Under 2PL-HP, a transaction that restarts or blocks another one may later suffer extensive delay or be restarted by other transactions when it tries to acquire locks on other data items. As a result, the transaction may miss its deadline, wasting the restart it executed or the waits caused to others. This intensifies the useless restarts and the useless waits problems for 2PL-HP.

Moreover, a high performance disk-based RTDBS uses deadline-cognizant techniques to schedule disk requests (Abbott and Garcia-Molina, 1990). Requests from transactions with earlier deadlines have a higher chance of being served first. The disk thus acts (indirectly) as an agent controlling the progress of the active

transactions. Under OCC-BC, the result is that higher priority transactions are advancing at a bit faster pace while lower priority ones are delayed (or blocked) at the disk access level. This abates the damage of restarts because: (1) the restarted transactions tend to be of lower priorities (i.e., later deadlines) and thus are more likely to survive the loss of slack due to the restarts, and (2) the slowly progressing low priority transactions tend to be restarted at an early stage and hence the loss of resource would be milder. Therefore, even though OCC-BC uses more restarts, the disk environment tends to reduce the cost of each.

Finally, under OCC-BC, a transaction gains possession of the “dirty” data pages in its write phase, during which any transaction whose read-set contains any of the dirty pages is refrained from *entering* the system. We can consider those conflicting transactions as being temporarily removed or suspended. In a disk-based environment, the duration of a transaction’s write phase could be substantial. When data contention is high, such as when the system is under a load stress, it is likely that a number of transactions are being removed for a non-trivial amount of time. This is particularly true in a multiprocessor environment in which there could be multiple transactions being in their write phases at the same time. In a high load situation, as we will see from our experiment result, moving some transactions out of resource contention can actually help the system to meet more transaction deadlines. This is because traditional real-time schedulers, such as EDF, fail miserably when the system is overloaded. Moreover, since the removed transactions do not hold any locks when they wait, the chance of data conflict is much reduced. This causes fewer restarts. OCC-BC thus indirectly imposes a load control mechanism in a disk-based system which improves system performance.

The situation in a memory-resident system is just the opposite to a disk-based system. Since data accesses are much faster and can be done in parallel, the adverse effects of useless restarts and useless waits are much reduced for 2PL-HP. Also due to parallel data accesses, under OCC-BC, low priority transactions are progressing at a similar pace as high priority ones at the data access level. This results in a higher probability that a high priority transaction being restarted by a lower one, and a higher probability that transactions are being restarted at a later stage. OCC-BC thus wastes more resources and performs worse than 2PL-HP. Moreover, fast data access also shortens the duration of transactions’ write phases. The effect of temporary removal of transactions under high load is much reduced. More transactions are thus allowed to compete resources at the same time. A higher level of concurrency causes more data conflict and transaction restarts. This works against OCC-BC in which too many restarts is its Achilles’ heel. In summary, the memory environment annuls most of

the advantages that OCC-BC enjoys in a disk-based system.

3. Two phase locking-lock writes all

With an understanding of the protocols' behavior and the data access characteristics of disk-based and memory-resident systems, our goal now is to devise a concurrency control protocol that performs well under various system environments: both memory-resident and disk-based RTDBS. Here, we propose the following variant of 2PL: 2PL-LW. The rules are:

1. A transaction is divided into a *read phase* and a *write phase*. All reads precede writes. In the read phase, all physical database operations are read operations. Any updates to the data items are written into the private workspace of a transaction instead of into the database immediately. In the write phase, permanent updates of the database will be performed by creating write operations to apply the updates from the private workspace to the database.
2. A data page can be locked under two modes: R-lock for reading and W-lock for writing. R-locks can be shared; W-locks are exclusive and are only acquired in the write phase.
3. When a transaction T tries to acquire an R-lock on a page that is being W-locked by another transaction, T waits; otherwise, T acquires the R-lock on the page and proceeds.
4. When a transaction T enters its write phase, it checks if any pages in its write set are W-locked by others. If none, T acquires all the W-locks, releases all the R-locks, and restarts all transactions that are holding R-locks on any pages in T 's write-set; Otherwise, T waits.
5. When a transaction T finishes, it releases all its W-locks. Transactions that are waiting for the corresponding data pages are rescheduled.

Similar to OCC-BC, under 2PL-LW, a transaction can restart others only when it is in its write phase, and therefore it will never be a target of restart itself. Thus, 2PL-LW generates much fewer useless restarts

than 2PL-HP does. Also, if a transaction T_A waits for another transaction T_B , T_B must be in its write phase, and thus it will never be blocked by others. Therefore, useless waits are avoided. Moreover, similar to 2PL-HP, 2PL-LW resolves conflicts by a combination of blocking (rules 3 and 4) and restarts (rule 4). 2PL-LW thus generates fewer restarts than OCC-BC, which uses a purely restart-based conflict resolution. Also, since unlocking of data pages occurs after locking of data pages in 2PL-LW, 2PL-LW is a 2 phase locking protocol and hence it is serializable. Finally, 2PL-LW can be shown (below) to be deadlock-free.

Lemma 1. *2PL-LW is deadlock-free.*

Proof. Assume that deadlock occurs under 2PL-LW. Without loss of generality, we assume that there are transactions $T_0, T_1, T_2, \dots, T_n$ such that T_0 waits T_1 , T_1 waits $T_2, \dots$, and T_n waits T_0 . Since T_0 waits T_1 , T_0 is trying acquire a lock (read or write) on a data page which is W-locked by T_1 . This implies that T_1 must be in its write phase and must have acquired all the W-locks (rule 4). It therefore will never have to wait for any transaction to release locks on data pages. A cyclic wait will never happen. 2PL-LW is thus deadlock-free.

4. Simulation models

To compare the performance of the concurrency control protocols, we model a disk-based RTDBS and a memory-resident RTDBS. Both models consist of the five components shown in Fig. 1. In brief, we have the following:

Source: The *Source* component generates a Poisson stream of transactions according to a specified system load. Attributes of a transaction, such as the number of data pages to be read and written, the amount of CPU processing required to process a data page, and the deadline, are generated according to some statistical distributions.

Transaction manager: The *Transaction Manager* models the execution of transactions. For example, in

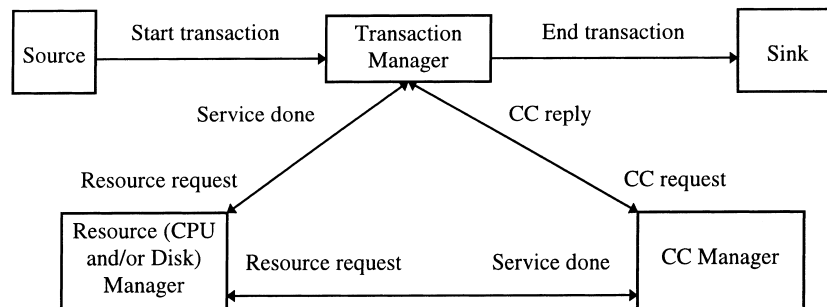


Fig. 1. RTDBS model.

2PL-HP, before a page is read, the transaction manager will send a request to the Concurrency Control Manager to acquire a lock, during which the transaction is suspended. It also assigns priorities to transactions and performs real-time scheduling. We assume a firm real-time system in which a tardy transaction is purged from the system by the transaction manager.

Concurrency control manager: The *Concurrency Control (CC) Manager* implements the details of the concurrency control protocols. For example, it maintains the lock table in 2PL-HP and checks for conflicting transaction when a committing transaction executes its write phase in OCC-BC. Four versions of the CC Manager are implemented in our simulator, modeling the different protocols under study: 2PL-HP, OCC-BC, 2PL-LW, and 2PL-LW2 (to be discussed in Section 6).

Resource manager: The *Resource Manager* models CPU and/or disk I/O resources. In particular, the resource manager handles disk scheduling for the disk-based RTDB model.

Sink: The *Sink* gathers various statistics about the system and transactions. These include: the number of restart, transaction restart point, CPU/disk utilization, transaction wait time, and system miss rate etc.

4.1. Disk-based RTDBS model

The disk-based model consists of a shared-memory multiprocessor system operating on disk-resident data. A single transaction queue is shared by the multiple processors. CPU scheduling is preemptive-resume EDF. There are *NumDisks* disks in the system, each with its own service queue. The disks are scheduled independently according to the priority-based variant of the Elevator Algorithm (Carey et al., 1989). We assume that all data are accessed from disk and ignore buffer pool considerations. The database itself is modeled as a collection of data pages and data accesses are done one page at a time. Disk access time is calculated by the formula:

Disk access time = *Seek Factor* $\times$ $\sqrt{n}$ + *DiskDelay*,

where n represents the number of tracks the disk head moves to service the request. In the simulation, we assume that disk requests are uniformly distributed across all disks and across all tracks within a disk.

Transactions are generated at the source as a Poisson process with a mean arrival rate of *ArrivalRate*. A transaction consists of a series of read accesses followed by a series of write accesses. A read access involves a concurrency control request to access permission, followed by a disk I/O to read the page, followed by a period of CPU usage to process the page. A write access is handled similarly except that the CPU processing time is omitted (Fig. 2). The page requests are generated from a uniform distribution spanning the entire database. The number of data pages read is uniformly distributed in the range $[0.5 \times \text{PageCount}, 1.5 \times \text{PageCount}]$. *PageCount* is the average number of pages accessed by a transactions. A page read by a transaction may be updated with a probability of *WriteProb*.

Each transaction has an associated deadline. We use the following formula for deadline assignment: $D_T = A_T + SF \times R_T$, where D_T is the deadline of transaction T , A_T the arrival time of transaction T , R_T the resource time of transaction T , and SF the slack factor.

The resource time of a transaction T is its response time if T is executed alone in the system (without encountering any contention for resources and data). The slack factor is a simulation parameter which controls how tight the deadlines are. We assume a *firm deadline* system. i.e., tardy transactions are discarded even if they are incomplete.

Table 2 summarizes the parameters in the simulation model.

4.2. Memory-resident RTDBS model

The memory-based model is similar to the disk-based model, except that disk requests are replaced by memory copy (between shared space and local space of transactions). We use the parameter *BCopyTime* to represent the copy time of a page. Fig. 3 illustrates the transaction model.

5. Simulation results

In this section we present our simulation results comparing the three concurrency control protocols:

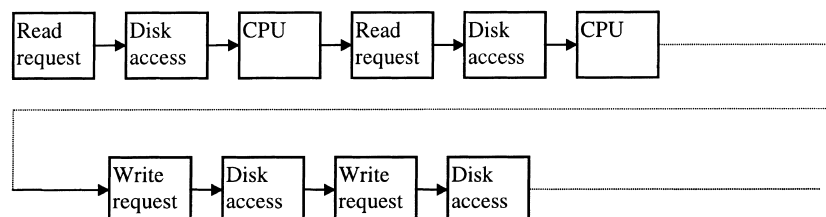


Fig. 2. Disk-based RTDBS transaction model.

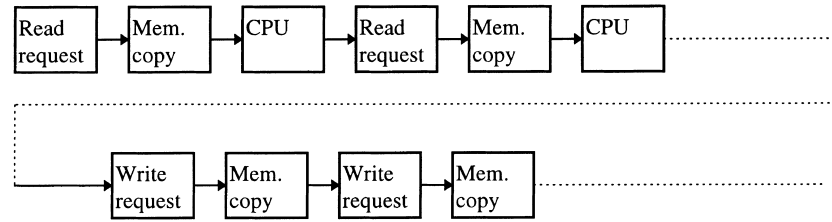


Fig. 3. Memory-based RTDBS transaction model.

2PL-HP, OCC-BC, and 2PL-LW in both disk-based and memory-resident RTDBS. Also shown in the graphs is a forth set of curves labeled 2PL-LW2. The protocol 2PL-LW2 is a slightly modified version of 2PL-LW, with which a simple admission control scheme is added. The purpose of such modification is to guard against system thrashes at sudden load surge. We will discuss 2PL-LW2 later in Section 6. We have done extensive experiments studying the various aspects of the protocols. Due to space limitation, our discussion here concentrates on the results which can illustrate the performance characteristics of the protocols. Interested readers are referred to Chiu and Kao (1996) for more details.

The simulation is written in the simulation language CSIM-18 (CSIM, 1996). At least 20,000 transactions are generated at each simulation run. We should point out that traditionally soft real-time systems are studied under high load situations. Most of the time, the system will hopefully operate under low load; no deadlines will be missed regardless of what scheduling or concurrency control protocol is used. However, once in a while the system will be overloaded, and it is precisely at those times when we need a protocol that can miss the fewest deadlines. For this reason, in order to highlight the protocols' performance difference, we model both the disk-based and memory-based systems under a moderate to high data contention and resource contention scenario. The baseline parameter settings are shown in Table 2.

Table 2
Simulation parameters and their baseline values

Parameter	Meaning	Baseline value
<i>ArrivalRate</i>	Poisson rate of transaction arrivals	5–40 tx/s
<i>TotalPage</i>	Number of pages in the database	1000 pages
<i>PageCount</i>	Average pages accessed/transaction	16 pages
<i>WriteProb</i>	Write probability/accessed page	0.25
<i>SlackFactor</i>	Slack factor in deadline assignment formula	4.0
<i>NumCPUs</i>	Number of processors	10
<i>NumDisks</i>	Number of disks	20
<i>NumDiskPrio</i>	Number of priority levels at each disk	5
<i>NumTracks</i>	Number of tracks per disk	1000
<i>PageCpuTime</i>	CPU time for processing data page	10 ms
<i>BCopyTime</i>	Page memory block copy time	0.5 ms
<i>DiskDelay</i>	Disk rotational + transfer delays	15 ms
<i>SeekFactor</i>	Factor relating seek time to distance	0.5 ms

5.1. Disk-based model

Fig. 4 shows the miss rate (the fraction of missed deadlines) of the three protocols in the disk-based environment as the transaction arrival rate ranges from 5 to 40 transactions per second. Similar to the observation discussed in (Haritsa et al., 1990), 2PL-HP misses more deadlines than OCC-BC does. The modified two-phase locking protocol, 2PL-LW, on the other hand, performs the best. Under the experiment setting, 2PL-LW outperforms 2PL-HP and OCC-BC by a wide margin when the load to the system is moderate (15–25 transactions/s). For example, at *ArrivalRate* = 20, 2PL-LW misses 7.8% of deadlines, which is about two times less than that of OCC-BC, and is about three times less than that of 2PL-HP. However, the performance advantage of 2PL-LW diminishes as the system load increases towards a high value. (This will be discussed in detail in Section 6.)

As we have discussed in Section 2, one of the major factors that affects the protocols' performance is the amount of system resources wasted in restarting transactions. Fig. 5 shows the number of restarts per

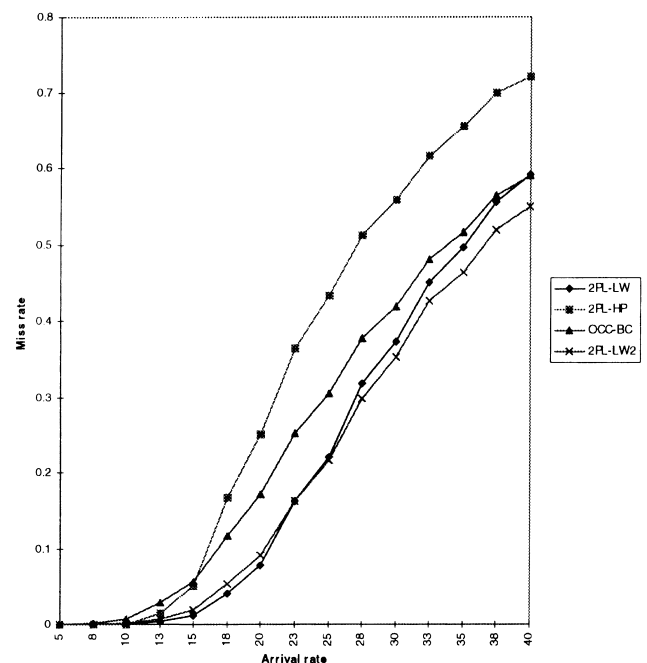


Fig. 4. Miss rate (Baseline disk model, low loading).

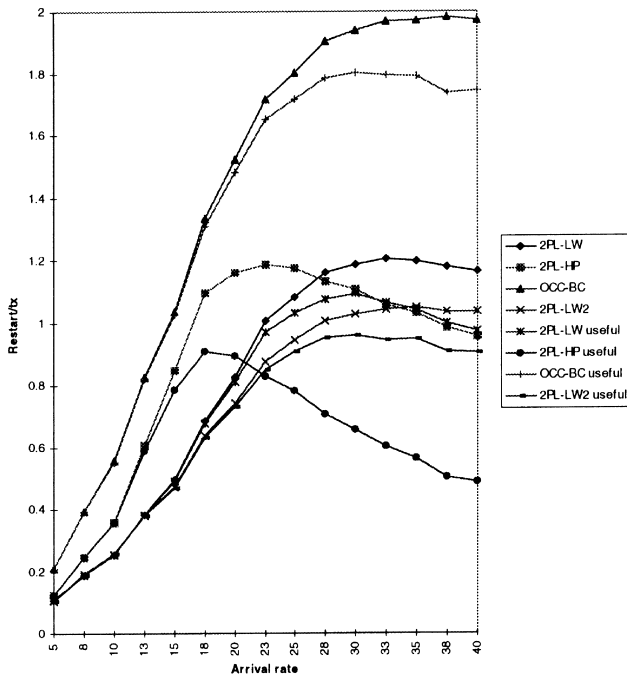


Fig. 5. Restart/tx (Baseline disk model, low loading).

transaction under the three protocols versus the system load. Also shown are the curves of *useful* restart per transaction (a restart is useful if the transaction that restarts another meets its deadline).

From the figure, we see that the general shape of the restart curves is ascending at low *ArrivalRate* and descending at high *ArrivalRate*. The reason is that at low load, as *ArrivalRate* increases, more transactions enter the system creating more conflict and restarts; At high load, however, system resources, like disk and CPU, are highly utilized. Queues start to build up at the resources and transactions experience long delay waiting for the resources. A transaction is thus likely to become tardy while it is waiting and is discarded before it conflicts with other transaction causing restarts. The restart curves thus go down at high value of *ArrivalRate*.

Comparing the restart curves of the protocols, we see that OCC-BC generates the highest number of restarts. This is due to the fact that OCC-BC detects data conflicts and executes restarts more liberally than others by checking read-sets and write-sets of transactions. However, the number of *useless* restarts are relatively small (i.e., the difference between the “restart” curve and the “useful restart” curve is relatively small). This is because only the transactions which have already entered its validation phase can restart others, and these transactions are very close to finishing. Also these transactions will not be restarted by or wait for other transactions once they enter their write-phases. 2PL-HP, on the other hand, generates fewer restarts than OCC-BC does, but a good number of these restarts are wasteful.

(The curves labeled “2PL-HP” and “2PL-HP useful” are pretty wide apart). Letting a transaction restart others when it is in the middle of its execution without any guarantee that it will complete therefore does not seem to do well. Finally, 2PL-LW generates few restarts and when it does, the restarts are usually useful. This is because 2PL-LW detects data conflicts by checking read-lock one at a time with the write-sets of other transactions, and restarts are done only by those transactions that are already in their write-phases. Also once a transaction enters its write-phase, it will not be restarted by or wait for other transactions.

To find out how much resource a transaction restart wastes, we measure the *restart point* of transactions (Haritsa et al., 1990). Fig. 6 shows, on average, the fraction of work a restarted transaction has completed. From the figure, we see that OCC-BC restarts transactions at a much earlier stage than 2PL-HP does. For example, at *ArrivalRate* = 30, under OCC-BC, a transaction is only 27% complete when it is restarted. Comparing this with the 54% completion under 2PL-HP, a restart caused by OCC-BC is about half as expensive as it is under 2PL-HP. The high restart rate of OCC-BC is thus offset by the low cost of a restart. As a combination of 2PL-HP and OCC-BC, the average transaction restart point for 2PL-LW falls in between that of the two basic protocols. For example, at *ArrivalRate* = 30, a restarted transaction under 2PL-LW is about 45% complete.

The effect of wasting system resources due to restarts is reflected by the system utilization. This is shown in Fig. 7 in which we compare the *useful CPU utilization*

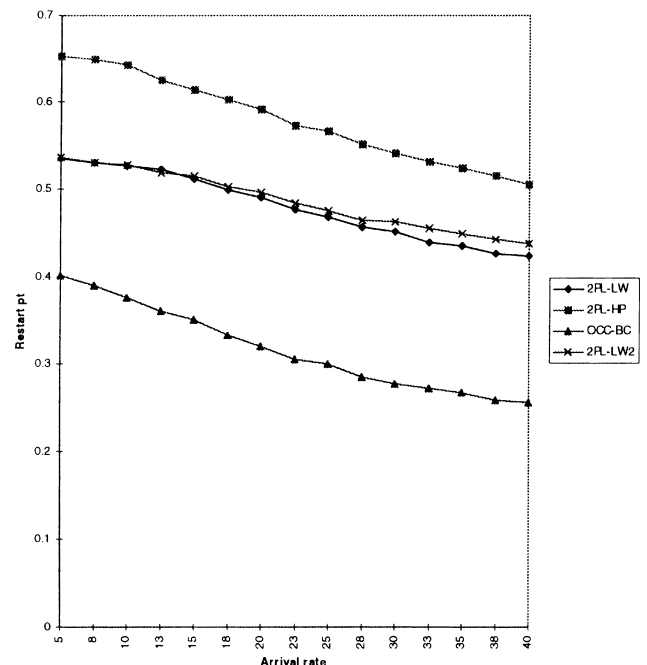


Fig. 6. Restart pt (Baseline disk model, low loading).

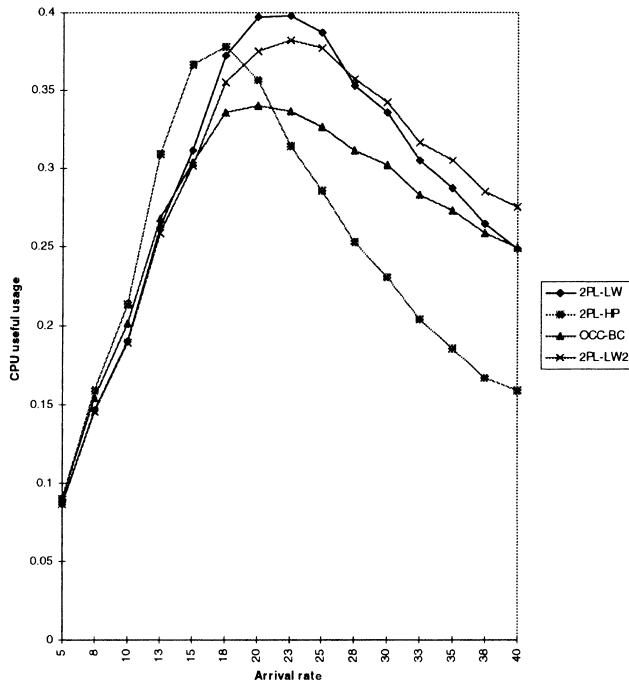


Fig. 7. CPU useful usage (Baseline disk model, low loading).

under the three protocols, i.e., the fraction of time the CPUs are used to serve transactions that make their deadlines. We see that the utilization curves start to go down when *ArrivalRate* is about 18–20 despite that more transactions are being pumped into the system. This indicates that the effect of data contention and restarts start to take their toll, wasting CPU cycles. From Fig. 4, the miss rate curves of the different protocols also start to diverge at *ArrivalRate* = 18. It is thus useful to compare the useful CPU utilization under the moderate to high load scenario. The fact that 2PL-HP makes the worse use of CPU cycles while 2PL-LW is the best within the range of interest helps explain the relative miss rates of the protocols indicated in Fig. 4.

The third performance factor is the amount of *useless wait*. Fig. 8 shows the average amount of data or lock waiting time that a transaction experiences in its lifetime. (For OCC-BC, a transaction experiences such wait when its read-set or write-set contains a data item that is marked “busy” by another transaction *T* executing in a write phase. The waiting transaction is not allowed to start within *T*’s write-phase, see Section 2.2.) Also shown in the figure are the curves labeled *useful*. A transaction’s wait is useful if the wait is caused by another transaction which meets its deadline; otherwise, the wait is considered useless.

From the wait curves we see that, in a disk-based environment, OCC-BC causes the largest among the wait time per transaction compared with the others. This difference is particularly marked when the system load is high (and thus when data contention is severe). This ob-

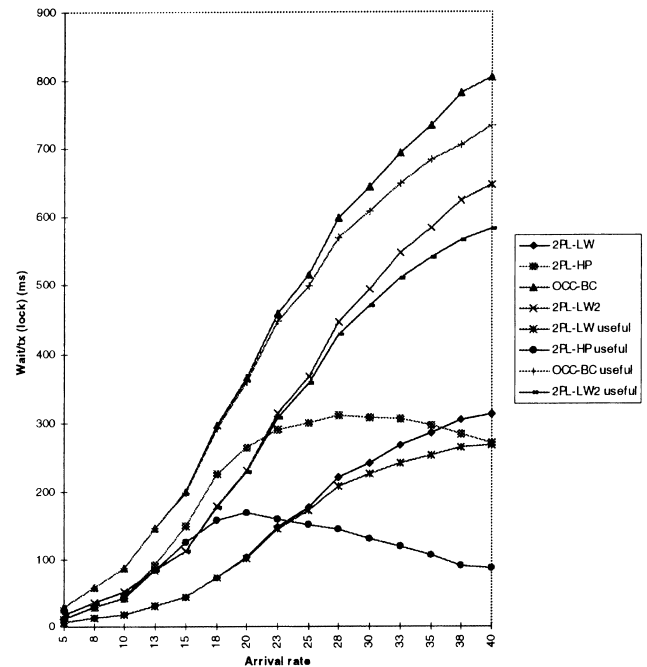


Fig. 8. Wait/tx (lock) (ms) (Baseline memory model, low loading).

servation can be attributed to the fact that slow and unpredictable data access results in protracted write phases. Although the amount of data-waiting time is large, most of the wait is useful under OCC-BC. Similar to the reasoning discussed on restarts, data-wait under OCC-BC is mostly useful because only the (finishing) transactions which have entered their write phases can block others. These transactions will not be restarted and are highly likely to make their deadlines. On the other hand, although 2PL-HP allows shorter data-wait, a large amount of the wait is useless. For example, when *ArrivalRate* = 40, only about one-third of the wait is useful. Long disk access time is again the culprit. A transaction that blocks others through locks may yet need to go through a number of reads and writes. Long data access time makes the transaction vulnerable to missing its deadline due to future delay and restarts, causing the wait it imposes on others wasteful.

Finally, the protocol 2PL-LW induces small amount of wait and most of the wait is useful. The short data-wait is because, like 2PL-HP, a transaction is blocked only *when* it needs to access a data item (while in OCC-BC, a transaction is blocked if a page it *will* access conflicts with some transaction’s write-set). Furthermore, most of the wait is useful because only finishing transactions (those in their write phases) can block others.

One may wonder why OCC-BC performs well (Fig. 4) even though it demands extensive data-wait (Fig. 8). We remark that while restarts clearly waste system resources, the performance impact of data-wait is more subtle. Under OCC-BC, a data-waiting transaction does not consume any system resources at all. It

does not hold any lock and thus does not compete with other active transactions. As long as the transaction has enough amount of slack to absorb the wait, it will be all right. As we will show later, blocking transactions in OCC-BC can in fact avoid system thrashes because it expedites the other active ones by reducing resource competition. On the other hand, under 2PL-HP, a blocked transaction could be holding locks. The longer the transaction waits, the higher the probability that (1) it blocks others forming a *chain of blocked transactions*; and (2) it gets restarted by others due to lock conflicts. Both factors are detrimental to the system's performance.

5.1.1. Memory-based model

We repeat the experiments comparing the three concurrency control protocols under the memory-resident RTDBS model. The result is presented in Figs. 9–13 showing the miss rates, (useful-) restarts, restart points, CPU useful utilization, and (useful-) wait time per transaction, respectively. Since disk delay does not play a part in the memory model, transaction execution is shorter and transactions are more likely to meet their deadlines. In order to highlight the performance differences of the protocols, in the figures, we show a higher loading environment (with *ArrivalRate* ranging from 25 to 60 transactions per second).

Interestingly, OCC-BC misses many more deadlines than the two locking protocols and 2PL-HP performs the best (Fig. 9). For example, when *ArrivalRate* = 45, OCC-BC misses 6% of transactions while 2PL-HP only misses 0.8%. The performance of 2PL-LW and 2PL-HP

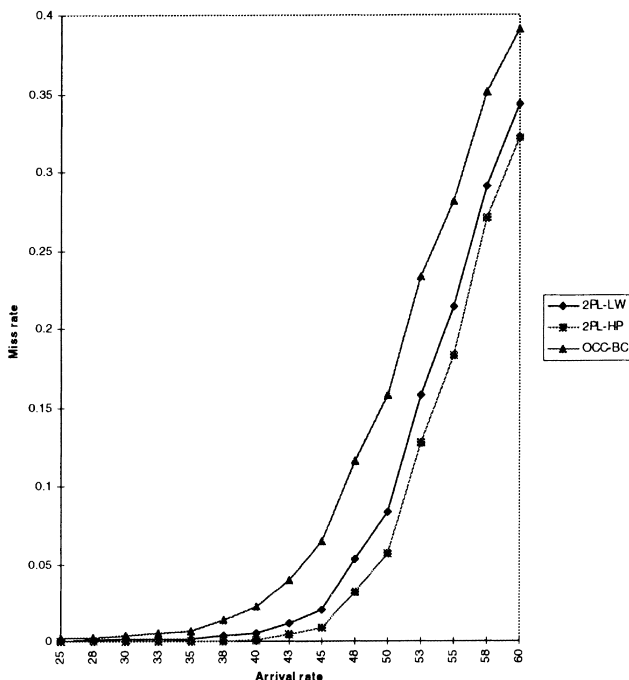


Fig. 9. Miss rate (Baseline memory model).

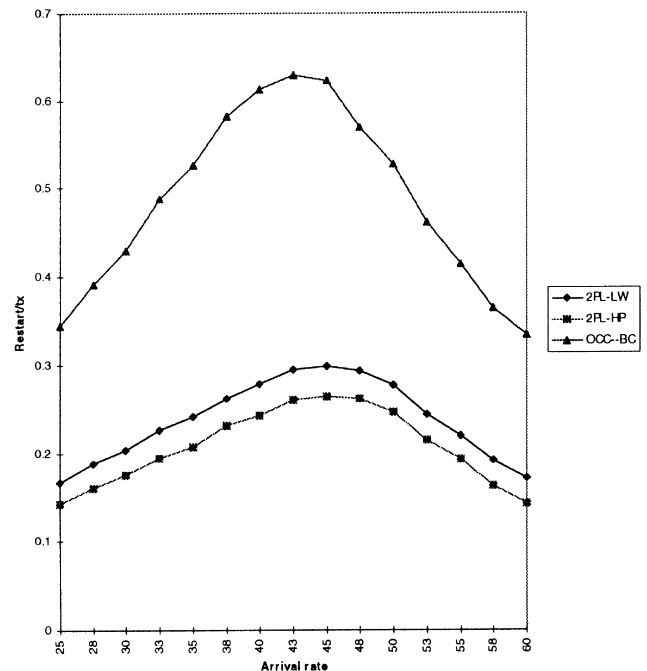


Fig. 10. Restart/tx (Baseline memory model).

are similar with 2PL-HP having a slight edge. To understand the relative performance of the protocols, we again revert to a study of the three factors: restarts, useless restarts, and useless wait.

Fig. 10 shows the average number of restarts per transaction under the three protocols. Similar to our observation in the disk-based model, OCC-BC restarts

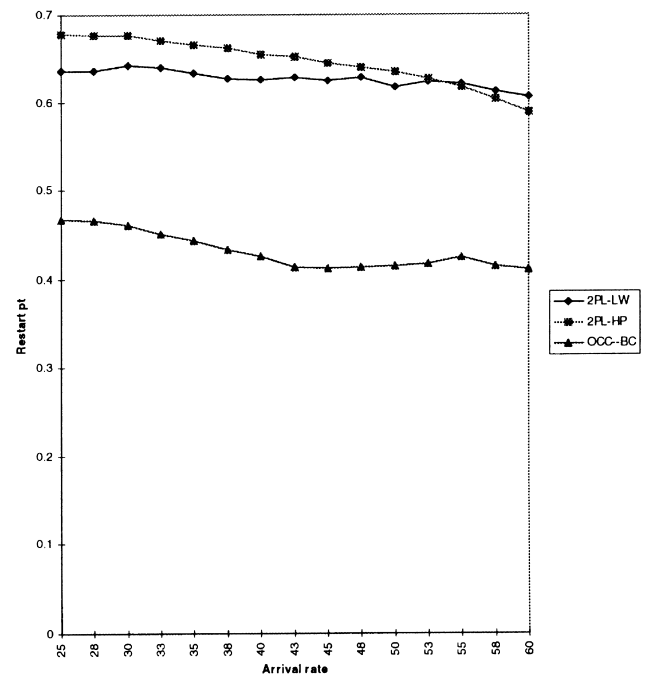


Fig. 11. Restart pt (Baseline memory model).

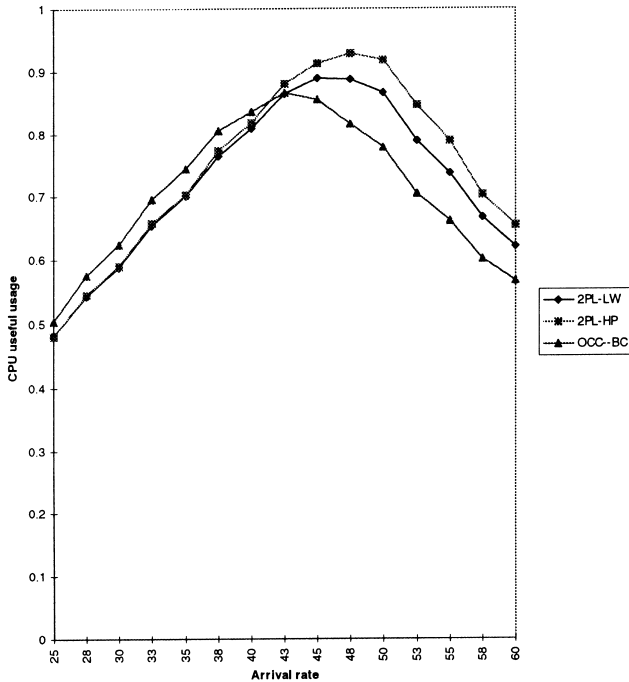


Fig. 12. CPU useful usage (Baseline memory model).

transactions more liberally resulting in the highest restart rate; Restarts in the lock-based protocols are more controlled and their restart rates are very similar. The major difference between the memory-based model and the disk-based model on restarts is the percentage of useless restarts. In the experiment we found that while

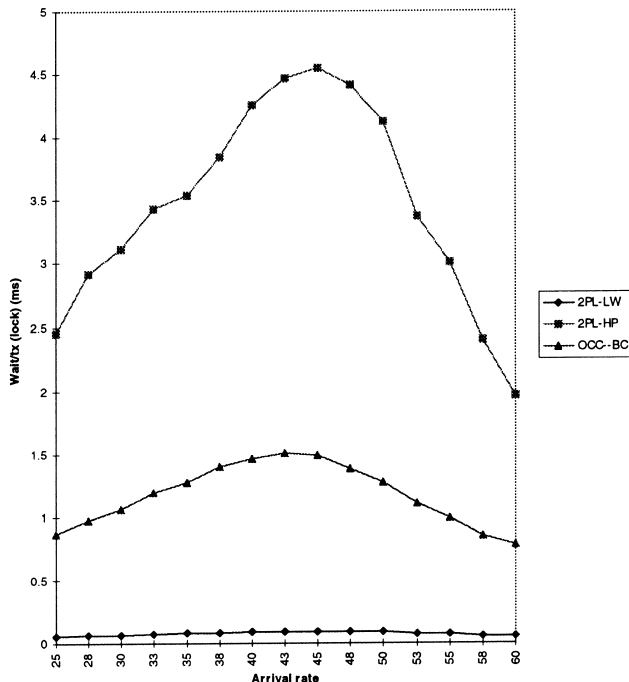


Fig. 13. Wait/tx (lock) (ms) (Baseline memory model).

a major portion of the restarts that occur under 2PL-HP are wasted in the disk-based environment, most of the restarts are *useful* in the memory case. In fact, the useful restart curves and the restart curves are so close that we remove the useful restart curves in Fig. 10 to improve readability. The reason for this phenomenon is that memory allows parallel and fast accesses. This eliminates much of the queuing at the data access level, and thus the high variability of data access time that happens in a disk-based system is no longer a problem in a memory-based system. Prompt services to data access requests reduce the amount of lock holding time, the probability of conflict and thus the number of restarts. A transaction that restarts others would be unlikely to experience extensive delay due to resource conflicts. It has a higher chance of meeting its deadline making the restarts it executed useful. Storing data in memory thus eliminates the biggest disadvantage (useless restarts) of 2PL-HP. Although the locking protocols are still restarting transactions at later restart points (on average, a transaction is more than 60% complete when it gets restarted in 2PL-HP or 2PL-LW, Fig. 11), the fewer number of restarts (Fig. 10) gives the locking protocol a better CPU utilization (Fig. 12).

Finally, Fig. 13 shows the average amount of data-wait per transaction. Although 2PL-HP causes the longest average wait time, the figures are insignificant compared with the total transaction execution time. In our memory-based model experiment, the average transaction execution time (without any data or resource contention) is about 200 ms. The highest amount of wait-time, as shown in Fig. 13, is only about 2.2% of a transaction execution time. The problem of useless wait is therefore an insignificant factor for 2PL-HP.

6. OCC-BC and load control

Careful readers might have noticed that in the disk-based model experiment, although the performance of 2PL-LW is the best among the three protocols under study (Fig. 4), it is being caught up by OCC-BC at high load. In this section we investigate how the relative performance of the protocols changes under a very high data contention situation. Here, let us stress that while we agree that a soft real-time system should not be designed to *operate* at such a high miss rate scenario under *normal* condition, it is useful to *study* such cases so that we can improve our protocols to save as many as deadlines as possible under *unexpected* system aberration.

We create a high data contention scenario in two ways: one by increasing the system work load and the other by increasing the transaction write probability. Fig. 14 shows the miss rates of the different protocols in the disk-based model as the transaction arrival rate ranges from 40 to 60.

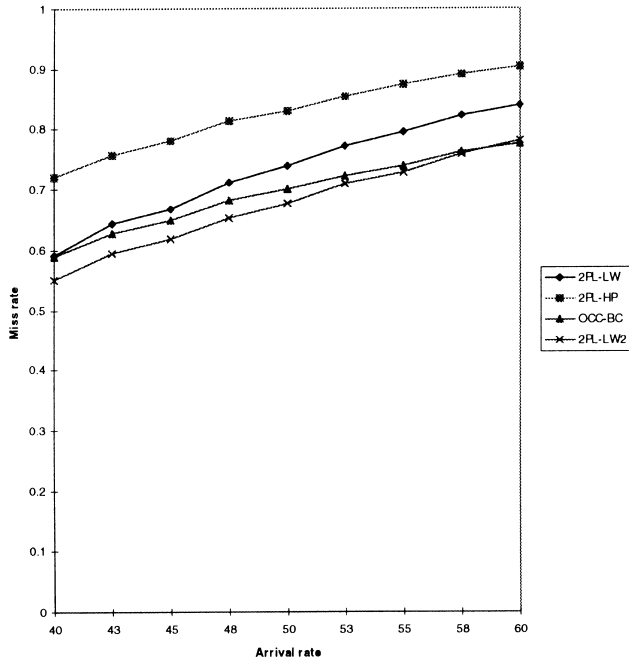


Fig. 14. Miss rate (Baseline disk model, heavy loading).

From the figure we see that under such a high load environment, OCC-BC has overtaken 2PL-LW as the better protocol, missing the fewest deadlines. The reason for this cross-over is that a higher system load implies more severe data contention. Under OCC-BC, when a transaction T enters its write phase, pages that are being updated are marked “busy”. All other transactions that conflict with T are restarted and are forbidden to enter the system competing for resources during T ’s write phase. This effectively regulates the population of active transactions in the system, expedites their executions, and allows them to meet their deadlines.

To support our claim, we measure the average wait time of transactions on CPU and on disk. The results are shown in Figs. 15 and 16, respectively. From the figures we see that transactions spend much shorter time in waiting for CPU and disk resources under OCC-BC than the two locking protocols. In fact, the average CPU queue length and the average disk queue lengths are much shorter for OCC-BC. Reducing the active population in an overloaded real-time system is beneficial to its performance, as it has been shown that EDF scheduler fails miserably under high load. We conclude that holding back some transactions helps improve the others’ chances of meeting deadlines, especially when the system is overloaded.

The idea of an implicit load control mechanism can be implanted into 2PL-LW by a simple modification of the rules. To imitate OCC-BC’s load control feature, we amend Rule 3 of 2PL-LW (see Section 3) as follows:

Modified Rule 3 of 2PL-LW:

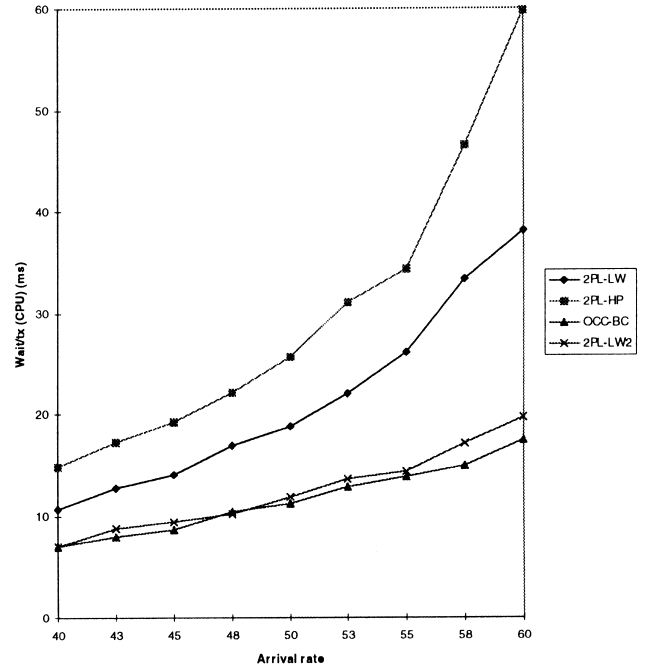


Fig. 15. Wait/tx (CPU) (ms) (Baseline disk model, heavy loading).

If a transaction T tries to acquire an R-lock on a page P in its read-set

If P is the first page T tries to read

Check if any pages in T ’s read-set is W-locked by another transaction.

If so, T waits; Otherwise, T acquires R-lock on P and proceeds.

If P is not the first page T tries to read

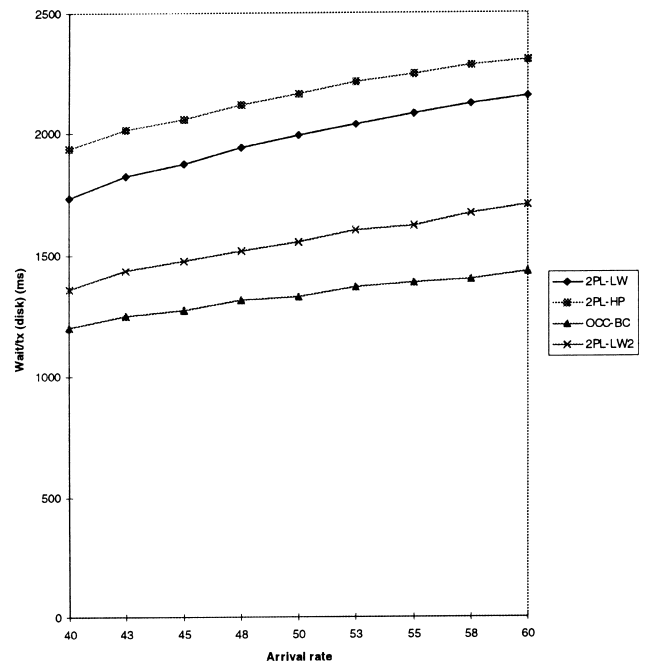


Fig. 16. Wait/tx (disk) (ms) (Baseline disk model, heavy loading).

Check if P is W-locked by another transaction.
If so, T waits; Otherwise, T acquires R-lock on P and proceeds.

We call the modified protocol **2PL-LW2**.

The difference between 2PL-LW2 and 2PL-LW is that under the modified protocol, before a transaction T acquires its first lock, the system checks if there is any potential conflict that T will have against other transactions due to T 's future lock requests. If so, the system suspends T early before it has acquired any locks. This helps avoid the bad effects of blocking a lock-holding transaction. Also, under a high load situation, when data contention is severe, 2PL-LW2 preserves system resources (CPU, data, and disk) to those transactions that are *likely* to get all the R-locks in their read-sets without blocking. These transactions thus have a good chance of meeting their deadlines.

The advantage of 2PL-LW2 can be seen from Figs. 4 and 14, in which 2PL-LW2 provides the best overall miss rate among the four protocols discussed. In particular, 2PL-LW2's performance is similar to 2PL-LW under low load, and is similar to OCC-BC under high load. The effect of load control as imposed by 2PL-LW2 can also be seen from Figs. 15 and 16 in which the average CPU and disk wait times under 2PL-LW2 are close to those exhibited by OCC-BC.

Our second experiment on high data contention scenario is done by increasing the transaction write probability to 1. That is, each page read is updated and is written back to disk. Fig. 17 shows the miss rate of the protocols as *ArrivalRate* ranges from 5 to 40. We

see that the performance cross-over point between 2PL-LW and OCC-BC occurs at a much lower arrival rate than in the baseline experiment. Again, a high data contention scenario demands a better load control mechanism. Protocol 2PL-LW2, which takes that into account gives the best overall performance. It is thus effective in meeting transaction deadlines in a high-load, high-contention situation.

7. Conclusions

In this paper we compared the performance of 2PL-HP and OCC-BC in both disk-based and memory-resident RTDBS. Although it was shown that OCC-BC performs better than 2PL-HP in disk-based systems, our study shows that due to the differences in the access characteristics between disk and memory, 2PL-HP outperforms OCC-BC in a memory-resident system. We indicated three important factors, namely, the number of restarts, useless restarts, and useless waits, which have great impact on the protocols' performance. Based on the analysis, we proposed a new lock-based protocol, 2PL-LW, which adopts the advantages of both 2PL-HP and OCC-BC. Our simulation results showed that 2PL-LW performed the best among the three protocols in a disk-based system, and comparably to 2PL-HP in a memory-resident system.

We have also investigated the protocols' behavior under a high load, high data contention scenario. We discover that an implicit load control mechanism exhibited by OCC-BC helps expediting the execution of those transaction that are likely to make their deadlines. This results in a reduced miss rate. We incorporate this mechanism into 2PL-LW. We show that the new protocol 2PL-LW2 performs the best, regardless of the loading condition.

Finally, since lock-based concurrency control protocols are more popular and more easily implemented, we conclude that 2PL-LW should be a better alternative to OCC-BC in multiprocessor real-time database systems.

References

- Abbott, R., Garcia-Molina, H., 1988. Scheduling real-time transactions: a performance evaluation. In: Proceedings of the 14th Conference on Very Large Database Systems. pp. 1–12.
- Abbott, R., Garcia-Molina, H., 1990. Scheduling I/O requests with deadlines: a performance evaluation. In: Proceedings of IEEE Real-Time System Symposium. pp. 113–124.
- Bernstein, P.A., Hadzilacos, V., Goodman, N., 1987. Concurrency Control and Recovery in Database Systems. Addison-Wesley, Reading, MA.
- Carey, M., Jauhari, R., Livny, M., 1989. Priority in DBMS resource scheduling. In: Proceedings of the 15th Conference on Very Large Database Systems.

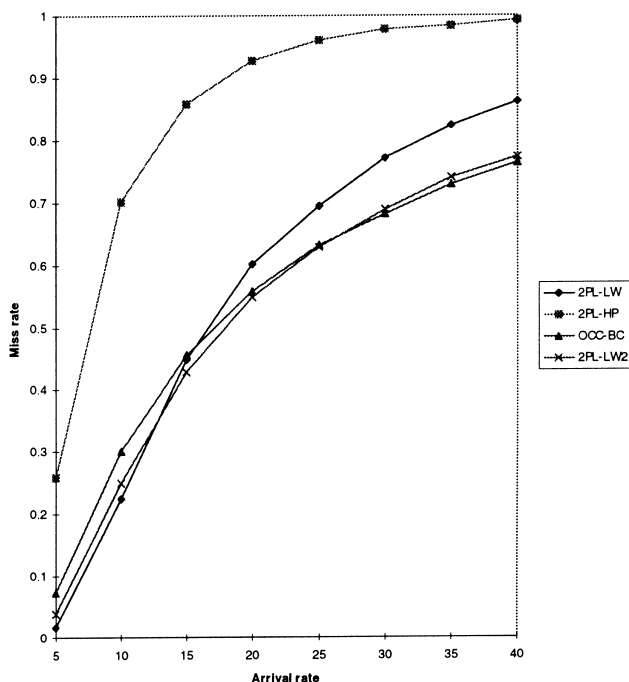


Fig. 17. Miss rate (Disk model, write prob = 1.0).

- Chiu, A., Kao, B., 1996. Comparing two-phase locking and optimistic concurrency control protocols in multiprocessor real-time database systems. Technical Report, TR-96-09, Department of Computer Science, The University of Hong Kong.
- CSIM, 1996. CSIM 18 User Guide, Mesquite Software, Inc. (URL: http://www.mesquite.com/1_intro.html).
- Haritsa, J.R., Livny, M., Carey, M., 1990. On being optimistic about real-time constraints. In: *Proceedings of the Ninth ACM Symposium on Principles of Database Systems*.
- Haritsa, J.R., Livny, M., Carey, M., 1991. Earliest deadline scheduling for real-time database systems. In: *Proceedings of IEEE Real-Time System Symposium*. pp. 232–242.
- Haritsa, J.R., Carey, M., Livny, M., 1993a. Data access scheduling in firm real-time system. In: *Proceedings of 13th International Conference on Distributed Computing Systems*. pp. 428–437.
- Haritsa, J.R., Livny, M., Carey, M., 1993b. Value-based scheduling in real-time database systems. *The VLDB Journal* 2 (2), 117–152.
- Huang, K., 1993. *Advanced Computer Architecture: Parallelism, Scalability, Programmability*, McGraw Hill, New York.
- Huang, J., Stankovic, J., Ramamritham, K., Towsley, D., 1992. Priority inheritance in soft real-time databases. *Journal of Real-Time Systems* 4 (3), 243–268.
- Huang, J., Stankovic, J., 1993. Experimental evaluation of real-time concurrency control schemes. *The VLDB Journal* 2 (2), 117–152.
- Menasce, D., Nakanishi, T., 1982. Optimistic versus pessimistic concurrency control mechanisms in database management systems. *Information Systems* 7 (1).
- Korth, H., Silberschatz, A., 1986. *Database System Concepts*, McGraw-Hill, New York, 1986.
- Robinson, J., 1982. Design of concurrency controls for transaction processing systems. Ph.D. Thesis, Carnegie Mellon University.
- Ulusoy, O., 1994. Processing of real-time transactions in a replicated database systems. *Journal of Distributed and Parallel Databases* 2 (4), 405–436.

Anthony Chiu is a Ph.D. candidate in computer science at the University of Hong Kong. He received his B.Sc and M.Phil. degrees in Mathematics from the University of Hong Kong in 1987 and 1991, and the M.Sc. degree with distinction in computer science from the City University of Hong Kong in 1995. He is now working as a Systems Manager in supporting IBM MVS systems for Information Technology Services Department (Hong Kong SAR Government). His research interests include real-time systems and operating systems.

Ben Kao received the BS degree in computer science from the University of Hong Kong in 1989 and the MS and Ph.D. degrees in computer science from Princeton University, Princeton, NJ in 1991 and 1995, respectively. He is currently an assistant professor in the Department of Computer Science at the University of Hong Kong. From 1989–1991, he was a teaching and research assistant at Princeton University. From 1992–1995, he was a research fellow in the Computer Science Department at Stanford University, Stanford, California. His research interests include database management, distributed algorithms, real-time systems, and information retrieval systems.

Kam-yiu Lam received the B.Sc.(Hons) degree on Computer Studies and Ph.D. degree from the City University of Hong Kong in 1990 and 1994, respectively. He is currently an Assistant Professor in the Department of Computer Science, the City University of Hong Kong. He has published more than 13 international journal papers and 25 conference papers. He has served as a program committee member of several international conferences on real-time systems, including the International Workshop on Real-time Databases, the International Workshops on Active, Real-time and Temporal Database Systems and the International Workshop on Real-time Systems and Applications. He is also a reviewer of some journals including the *IEEE Transactions on Computer*, *IEEE Transactions on Data and Knowledge Engineering*, *The Computer Journal*, the *Information Processing Letter* and the *Journal of Systems and Software*. His current research interests are real-time database systems, real-time active database systems, mobile computing and distributed multimedia systems. He is the Guest Editor of the *Journal of Systems and Software* on the special issue of Real-time Active Database Systems. Dr. Lam is a member of the IEEE and he is also an executive member of the ACM Hong Kong Chapter.